

Kalimati Market Analytics System: Final Project Report

Link to the Codebase: <https://github.com/ppeggyy/kalimati-market-analytics-system>

Link to the Website: <https://kalimati-frontend-deploy.vercel.app/>

1. Summary

The **Kalimati Market Analytics System** is a full-stack, cloud-deployed decision-support platform designed to track, analyze, and forecast agricultural commodity prices from Nepal's Kalimati Market. By integrating automated data engineering, optimized relational databases, complex backend analytics, machine learning, and an interactive React frontend, the system provides stakeholders with real-time risk assessment, historical trend analysis, and predictive pricing intelligence.

2. System Architecture & Cloud Deployment

The system is built on a highly decoupled, modern **five-layer modular architecture**, deployed entirely on scalable cloud infrastructure.

Cloud Deployment (Orchestrated by Subrna Thakurathi)

The application leverages a distributed, serverless architecture to ensure high availability and performance:

- **Frontend (Vercel):** The React-based user interface is deployed on Vercel, which provides a globally distributed Edge Network for sub-second page loads. Vercel automatically builds the application via Vite.
 - **Backend API (Render):** The FastAPI Python backend is deployed as a web service on Render. It is connected directly to the GitHub repository, facilitating Continuous Integration and Continuous Deployment (CI/CD) pipelines for automatic redeployments upon code pushes.
 - **Database (Neon Serverless Postgres):** The PostgreSQL database is hosted on Neon. This serverless database architecture separates compute from storage, allowing it to scale resources on-the-fly to handle heavy analytical query loads while maintaining strict security.
-

3. How the System Works (Core Functionalities)

A. Data Engineering & ETL Pipeline

The system automates the ingestion of over 102,000 raw market records. Missing data (such as weekends or market holidays) is handled using a tiered imputation algorithm: **forward-fill** is applied for gaps of 1-3 days, and **linear interpolation** is used for gaps of 4-14 days. The pipeline normalizes the data and performs incremental database updates, reducing daily sync times from 8 minutes to 2 minutes.

B. High-Performance Analytics

The system ranks 147 unique commodities by market volatility (standard deviation), detects sudden price spikes using rolling-window averages, and calculates market spreads. To handle these heavy mathematical operations without lagging the UI, the backend uses PostgreSQL JSONB caching to store and retrieve pre-computed metrics in milliseconds.

C. Machine Learning Forecasting

The predictive engine utilizes a statistical **ARIMA (2, d, 1)** model to forecast prices 30 days into the future. It dynamically runs an Augmented Dickey-Fuller (ADF) test to ensure the mathematical stationarity of the pricing data before fitting the model. Predictions are returned alongside a 95% confidence interval band and are continuously monitored using MAE, RMSE, and MAPE error metrics.

D. Interactive UI

The frontend visualizes the data through interactive Recharts, including line charts, seasonal area charts, and risk-comparison bars, all housed within a responsive Single-Page Application (SPA).

4. Codebase Overview

The codebase is logically separated into three main repositories/folders:

I. Data Engineering (/data_engineering)

- `data_extraction_pipeline.py`: Contains the `DataPipeline` class responsible for automated data fetching, cleaning, tiered imputation, and incremental database loading.

II. Backend (/kalimati_backend-main)

- `app/main.py`: The FastAPI application factory and CORS middleware configuration.
- `app/api/prices.py` & `app/api/analytics.py`: The routing layers containing 9 RESTful endpoints for historical, seasonal, and volatile data delivery.
- `app/core/analytics.py`: The core mathematical engine utilizing Pandas and NumPy to compute standard deviations, moving averages, and spikes.
- `app/schemas/price.py`: Pydantic schemas enforcing strict data validation for API requests and responses.
- `app/services/forecast_service.py`: Contains the ARIMA machine learning logic, ADF tests, and error metric calculations.
- `app/db/models.py` & `app/db/bootstrap.py`: SQLAlchemy ORM definitions and the raw SQL logic that provisions B-Tree indexes and the `analytics_cache` table.

III. Frontend (/kalimati_frontend-main)

- `src/App.jsx`: The React application layout and router.

- `src/api.js`: The central API client handling asynchronous HTTP requests to the Render backend.
 - `src/pages/Dashboard.jsx` & `Forecast.jsx`: The view components combining React state management with Recharts data visualizations.
-

5. Team Roles & Contributions

The success of this comprehensive system was achieved through strict modularization and dedicated team roles:

1. Subrna Thakurathi (Project Lead, Data Engineer, & System Architect)

- **System & Cloud Architecture:** Designed the 5-layer decoupled architecture and successfully executed the full-stack cloud deployment across Vercel, Render, and Neon.
- **ETL Pipeline:** Built the automated Python `DataPipeline` that extracted 102,550 raw rows and populated the database with over 160,000 clean records.
- **Data Imputation:** Designed and implemented the tiered imputation logic (forward-fill and linear interpolation) to resolve missing holiday data.

2. Robinson Ghimire (Backend & Optimisation Engineer)

- **FastAPI Layer:** Built the high-performance communication layer, CORS middleware, and 9 RESTful API endpoints.
- **Analytics Engine:** Programmed the core mathematical logic for volatility rankings, trend tracking, and price spike detection.
- **Data Validation:** Implemented strictly typed Pydantic schemas to validate all data entering and leaving the system.

3. Ayush Bist (Frontend & Visualisation Developer)

- **React Dashboard:** Designed and built the responsive single-page application using React and Vite.
- **Data Visualization:** Implemented interactive UI components and complex Recharts visualizations, mapping raw JSON arrays to visual trend lines, areas, and pie charts.
- **API Wiring:** Built the central API client to asynchronously connect the frontend interface to the live production database.

4. Nikit Shrestha (Analytics & ML Lead)

- **Predictive Modeling:** Developed and integrated the ARIMA(2, d, 1) forecasting model to predict 30-day future commodity prices.
- **Statistical Validation:** Implemented dynamic Augmented Dickey-Fuller (ADF) tests to enforce data stationarity.

- **Error Monitoring:** Built the mathematical tracking logic for MAE, RMSE, and MAPE to ensure predictions maintain a targeted error rate of $\leq 15\%$.

5. Shashi Thapa Magar (Database Developer)

- **Schema Design:** Designed the fully normalized PostgreSQL database architecture and implemented the SQLAlchemy ORM mapping.
- **Performance Optimization:** Provisioned crucial B-Tree indexes on product names and dates to achieve sub-3-second query times across 160,000 rows.
- **Analytical Caching:** Engineered an advanced caching mechanism using PostgreSQL's JSONB data type to store pre-calculated heavy analytics, reducing load times to milliseconds.